

PROJECT ATTEMPT 2

Project Results

Review your submission outcome, score components, and peer feedback.

See all projectsProject statistics

TOTAL SCORE	STATUS	REVIEWED ENOUGH PEERS
28	Passed	Yes
Criteria Breakdown		
Retrieval evaluation2		
No evaluation of retrieval is provided (0 points)0 votes		
Only one retrieval approach is evaluated (1 point)0 votes		
Multiple retrieval approaches are evaluated, and the best one is used (2 points)2 votes		
RAG evaluation2		
No evaluation of RAG is provided (0 points)0 votes		
Only one RAG approach (e.g., one prompt) is evaluated (1 point)1 vote		
Multiple RAG approaches are evaluated, and the best one is used (2 points)1 vote		
Interface2		
No way to interact with the application at all (0 points)0 votes		
Command line interface, a script, or a Jupyter notebook (1 point)0 votes		
UI (e.g., Streamlit), web application (e.g., Django), or an API (e.g., built with FastAPI) (2 points)2 votes		
Ingestion pipeline2		
No ingestion (0 points)0 votes		
Semi-automated ingestion of the dataset into the knowledge base, e.g., with a Jupyter notebook (1 point)1 vote		
Automated ingestion with a Python script or a special tool (e.g., Mage, dlt, Airflow, Prefect) (2 points)1 vote		
Monitoring2		
No monitoring (0 points)0 votes		
User feedback is collected OR there's a monitoring dashboard (1 point)0 votes		
User feedback is collected and there's a dashboard with at least 5 charts (2 points)2 votes		
Containerization2		
No containerization (0 points)0 votes		
Dockerfile is provided for the main application OR there's a docker-compose for the dependencies only (1 point)0 votes		
Everything is in docker-compose (2 points)2 votes		
Problem description2		
The problem is not described (0 points)0 votes		
The problem is described but briefly or unclearly (1 point)0 votes		
The problem is well-described and it's clear what problem the project solves (2 points)2 votes		
RAG flow2		
No knowledge base or LLM is used (0 points)0 votes		
No knowledge base is used, and the LLM is queried directly (1 point)0 votes		
Both a knowledge base and an LLM are used in the RAG flow (2 points)2 votes		
Reproducibility2		
No instructions on how to run the code, the data is missing, or it's unclear how to access it (0 points)0 votes		
Some instructions are provided but are incomplete, OR instructions are clear and complete, the code works, but the data is missing (1 point)0 votes		
Instructions are clear, the dataset is accessible, it's easy to run the code, and it works. The versions for all dependencies are specified. (2 points)2 votes		
Best practices1		
Hybrid search: combining both text and vector search (at least evaluating it) (1 point)2 votes		
Document re-ranking (1 point)0 votes		
User query rewriting (1 point)0 votes		
Bonus points0		
Deployment to the cloud (2 points)0 votes		
Bonus point (1 point)0 votes		
Bonus point (1 point)0 votes		
Bonus point (1 point)0 votes		

Score Components	
Project score	19
Peer review score	9
Project LIP	0
Peer review LIP	0
FAQ contribution	0

Submission	
Project link	https://github.com/soumen7saha/comprehensive-medical-qna-assistant
Commit ID	de7f5a3
Submitted at	Aug. 15, 2026, 1:55 p.m.

Evaluation Feedback	
PEER FEEDBACK	Excellent submission. Retrieval evaluation is thorough — comparing text, vector, and hybrid (RRF) search with hit rate/MRR on a 867-query ground truth set, and picking hybrid as the winner, is exactly the kind of rigor this criterion rewards. RAG evaluation judges good/bad but doesn't compare alternative prompts. Best practices: hybrid search only — re-ranking or query rewriting would be a natural next step. Nice that the dataset is bundled in the repo, which made reproducibility straightforward to confirm.
PEER FEEDBACK	Strong and well-structured RAG project. The README is clear and well written, with helpful architecture details, screenshots, and demo videos. Retrieval evaluation is particularly strong, comparing text, vector, and hybrid search and using the best-performing approach. The Streamlit UI, monitoring dashboard, feedback collection, and Docker setup are also well implemented. The main improvement would be evaluating multiple RAG approaches, such as different prompts or configurations, instead of only the final pipeline. The ingestion script could also be made directly executable as a complete ingestion pipeline.